

Chapter 9 - Summary and Sample Questions

Concept-focused summary of training and adapting vision-language models

Chapter 9 presents vision-language model adaptation as a **ladder of interventions** applied to a common autoregressive policy. A VLM is written as

$$p_{\theta}(y \mid x, v) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x, v),$$

where x is the text instruction, v is the visual input, and y is the generated response. This factorization is deliberately kept fixed across the chapter so that each method can be understood by asking one clean question: **what exactly is being changed?**

The chapter’s central conceptual distinction is between methods that change only the **conditioning context** and methods that change the **policy parameters**. Zero-shot and few-shot prompting keep θ fixed. Zero-shot changes only the prompt, while few-shot inserts demonstration examples into the context window. The key idea is that behavior can still change dramatically without gradient descent because the transformer’s hidden states depend on the full context. So few-shot learning is not parameter learning; it is **context-conditioned temporary adaptation**.

RAG adds a second kind of inference-time adaptation by introducing an **external memory**. Instead of depending only on pretrained knowledge or context examples, the model retrieves documents and conditions generation on them. In probabilistic form,

$$p_{\theta,\eta}(y \mid x, v) = \sum_{z \in \mathcal{Z}} p_{\eta}(z \mid q(x, v)) p_{\theta}(y \mid x, v, z).$$

This makes RAG a latent-variable model over retrieved evidence. The chapter emphasizes that the key object is not just the retrieved top- k list, but the posterior responsibility weight

$$w(z) = \frac{p_{\eta}(z \mid q(x, v)) p_{\theta}(y^* \mid x, v, z)}{\sum_{z'} p_{\eta}(z' \mid q(x, v)) p_{\theta}(y^* \mid x, v, z')},$$

which tells us which document actually helped explain the gold answer. This is important because it separates **retrieval plausibility** from **explanatory usefulness**.

The chapter then moves from inference-time adaptation to **parameter learning**. In supervised fine-tuning, the model is trained on labeled triples (x_i, v_i, y_i^*) and optimized by next-token maximum likelihood:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{i=1}^N \sum_{t=1}^{T_i} m_{i,t} \log p_{\theta}(y_{i,t}^* \mid y_{i,<t}^*, x_i, v_i).$$

Here the mask $m_{i,t}$ determines which tokens count toward the loss. The chapter stresses an important multimodal instruction-tuning idea: in chat-style training, one often wants to predict only the **assistant span**, not the user or system text. So the practical question is not merely “what is the target answer?” but also “which token positions should define the learning signal?”

LoRA and QLoRA are presented as changes in **parameterization**, not changes in the task objective. LoRA keeps the pretrained weight matrix W frozen and learns a low-rank update

$$W' = W + \Delta W, \quad \Delta W = \frac{\alpha}{r} B A.$$

This reduces trainable parameters from $d_{\text{out}} d_{\text{in}}$ to roughly $r d_{\text{in}} + d_{\text{out}} r$. The key conceptual lesson is that SFT and LoRA are not competing methods at the same level. SFT defines **what objective is optimized**, while LoRA defines **which parameters are allowed to move and in what form**. The chapter also explains why adapting attention projections such as `q_proj`, `k_proj`, `v_proj`, and `o_proj`, together with MLP projections such as `gate_proj`, `up_proj`, and `down_proj`, changes both information routing and nonlinear transformation.

The alignment part of the chapter begins with **preference tuning** through DPO. Instead of imitating one gold answer, DPO compares a preferred response y_w with a rejected response y_l and encourages the policy to increase the preferred answer’s relative log-probability compared with a frozen reference policy:

$$\mathcal{L}_{\text{DPO}}(\theta) = - \log \sigma \left(\beta \left[\log \pi_{\theta}(y_w \mid x, v) - \log \pi_{\theta}(y_l \mid x, v) - \log \pi_{\text{ref}}(y_w \mid x, v) + \log \pi_{\text{ref}}(y_l \mid x, v) \right] \right).$$

The conceptual appeal is that DPO performs preference alignment without explicit online rollouts or PPO-style advantage estimation. But the chapter also warns that **relative preference is not identical to factual correctness**. If the preference pairs reward style more than truth, the model may become better aligned to human taste while becoming worse grounded scientifically.

The last stage reframes VLM adaptation as reinforcement learning. A state is the prompt, image, and generated prefix; an action is the next token; a trajectory is the full answer. The classical policy-gradient objective is written in terms of expected trajectory reward, and PPO is discussed as the standard trust-region intuition. For the runnable VLM setting, the chapter emphasizes RLOO, where several completions are generated for the same prompt, each is scored, and each completion is compared to the average reward of the others:

$$b_i = \frac{1}{G-1} \sum_{j \neq i} r_j, \quad A_i = r_i - b_i.$$

The simplified loss is

$$\mathcal{L}_{\text{RLOO}}(\theta) = - \frac{1}{G} \sum_{i=1}^G A_i \log \pi_{\theta}(o_i \mid q).$$

This makes RLOO conceptually attractive because it uses a **relative baseline** for the same prompt and avoids some of the machinery of heavier PPO-style pipelines.

The broad lesson of the chapter is that adaptation methods differ along several axes at once:

- whether they change context or weights,
- whether they inject knowledge, demonstrations, labels, preferences, or rewards,
- whether they are temporary or persistent,
- whether they optimize imitation, preference ranking, or reward,
- and how much data, compute, and evaluation burden they require.

That is why the chapter ends with a budget ladder. Zero-shot and few-shot are cheap but limited. RAG adds grounding without changing the model. SFT and LoRA create reusable specialization. DPO aligns relative preferences. RL-based methods are the most expressive, but also the most expensive and easiest to misuse if the reward is poorly defined.

Sample Final Exam Questions

Question 1

Why is the chapter’s adaptation ladder best understood as a sequence of **different intervention types** rather than a simple ranking from weak to strong methods?

Sample answer

The methods differ along multiple conceptual dimensions, not just “strength.”

Zero-shot, few-shot, and RAG mainly change the **conditioning information** supplied at inference time, while SFT, LoRA, DPO, and RL-based methods change the **policy itself**.

This means the methods intervene at different levels:

- zero-shot changes only the instruction,
- few-shot changes the context by adding demonstrations,
- RAG changes the context by adding retrieved evidence,
- SFT changes the objective over labeled outputs,
- LoRA changes the parameterization of the update,
- DPO changes the supervision signal from demonstrations to preferences,
- RL-based methods optimize expected reward.

So the ladder is not merely a scale of “more powerful.” It is a structured map of **what is being modified**: prompt, memory, labels, parameterization, preferences, or rewards. That is why the methods are often complementary rather than mutually exclusive.

Question 2

Why does the chapter emphasize adapting both attention projections and MLP projections in transformer-based VLMs?

Sample answer

The chapter highlights that different sublayers play different conceptual roles.

Attention projections such as W_Q , W_K , W_V , and W_O affect **routing**: they determine what information is queried, matched, attended, and reinserted into the residual stream. MLP projections such as `gate_proj`, `up_proj`, and `down_proj` affect **transformation**: they determine how information is nonlinearly processed after it arrives.

So adapting only attention changes how information flows, while adapting only the MLP changes how information is reshaped. Many downstream tasks need both:

- new visual-semantic associations,
- new token-to-token interaction patterns,
- and new local feature transformations.

That is why the chapter presents a broader adapter target set rather than treating LoRA as only an attention modification.

Question 3

How does the chapter reinterpret classical reinforcement-learning notation for autoregressive VLM generation?

Sample answer

The chapter shows that RLHF does not require abandoning the language-model viewpoint. It simply reinterprets token generation in RL terms:

- state s_t : prompt, image, and generated prefix,
- action a_t : next token,
- policy $\pi_{\theta}(a_t \mid s_t)$: next-token distribution,
- trajectory τ : full generated answer,
- reward: scalar score assigned to the answer.

This mapping matters because it explains why policy-gradient ideas apply naturally to VLMs. The model is still generating one token at a time, but now the sequence is judged by a reward function, human preference model, or verifier.

So RLHF is best seen as **policy optimization over token sequences conditioned on multimodal context**.

Question 4

Suppose a VLM gives well-formatted answers but repeatedly confuses closely related species because it lacks domain knowledge. Why would RAG often be a more principled first intervention than SFT or DPO?

Sample answer

If the primary failure is **missing knowledge**, then the problem is not yet one of answer style or preference alignment. RAG directly addresses the missing-knowledge bottleneck by injecting external evidence at inference time.

Conceptually, RAG changes the conditional distribution from

$$p_{\theta}(y \mid x, v)$$

to a retrieved-evidence version such as

$$p_{\theta,\eta}(y \mid x, v) = \sum_z p_{\eta}(z \mid q(x, v)) p_{\theta}(y \mid x, v, z).$$

This is often more principled than immediate SFT because SFT teaches the model to imitate desired outputs, but it does not necessarily give it access to new, explicit supporting evidence at inference time. DPO is even less direct for this failure mode because it adjusts preferences among answers, not the factual evidence base itself.

So when the central issue is **grounding and missing task knowledge**, RAG is often the cleanest first upgrade.

Question 5

How can RAG, LoRA-based SFT, and RL-based alignment be viewed as operating on three different layers of adaptation within one overall system?

Sample answer

These methods operate at different levels of the stack.

RAG acts on the **evidence layer**. It decides what external information is available to the generator at inference time.

LoRA-based SFT acts on the **policy-parameter layer**. It changes the model’s internal mapping from multimodal context to answer distribution, but does so through a low-rank parameterization.

RL-based alignment acts on the **behavioral objective layer**. It changes what the policy is rewarded for, typically using preference or reward signals rather than fixed gold outputs.

So one can think of the three as:

- RAG: change what the model sees,
- LoRA/SFT: change how the model maps seen information to outputs,
- RL alignment: change what output behavior is incentivized.

This layered view explains why the methods can be composed rather than treated as mutually exclusive alternatives.